

Example (Section 11.3 in Ruppert and Matteson)

The **forward rate function** $r(t)$ returns the current forecast of bond interest rates at time t in the future. The current price of a zero-coupon bond with maturity T can be found by discounting the face value of the bond by the forward rates “averaged” over the period until maturity:

$$P(T) = \text{Face Value of Bond} \times \exp\left(-\int_0^T r(t) dt\right)$$

(The face value of the bond is the amount to be paid to the holder at maturity.)

Simple derivation shows that

$$r(T) = -\frac{\partial}{\partial T} \log P(T).$$

Hence, we can estimate $r(t)$ using current zero-coupon bond prices.

In particular, consider a grid of observable maturity dates $T_0 < T_1 < \dots < T_n$ and approximate the derivative of $\log P(T)$ at T_i using

$$\frac{\log(P(T_i)) - \log(P(T_{i-1}))}{T_i - T_{i-1}}$$

for $i = 1, 2, \dots, n$.

This sets up a natural regression problem where the response is

$$Y_i = -\frac{\log(P(T_i)) - \log(P(T_{i-1}))}{T_i - T_{i-1}}$$

and our simple model assumes that

$$Y_i = r(T_i) + \epsilon_i$$

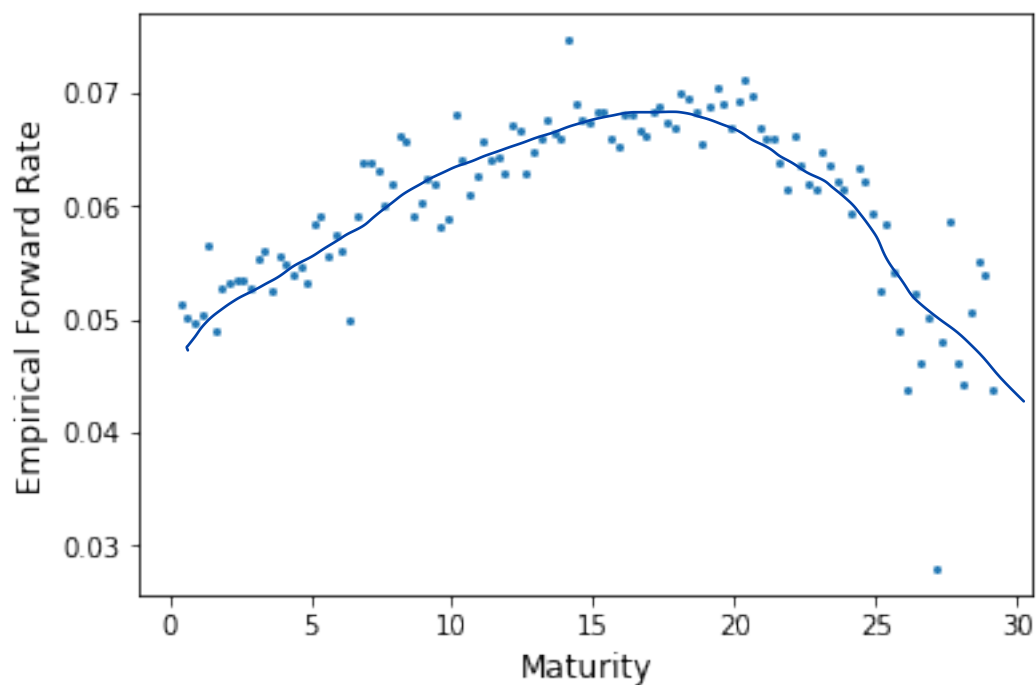
where the ϵ_i are iid, normally distributed, with mean zero. Hence, in this example we have a single covariate, time to maturity (T).

The data we have can be read in and visualized as follows:

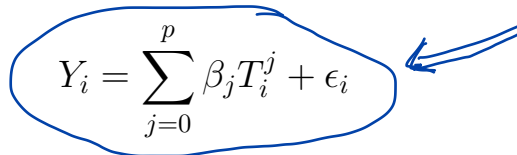
```
In [11]: import pandas as pd
```

```
forratedat = pd.read_csv(  
    "http://www.stat.cmu.edu/~cschafer/MSCF/forratedat.txt",  
    sep=" ")
```

```
fig, axs = plt.subplots(1, 1, figsize=[6,4])  
axs.plot(forratedat['maturity'], forratedat['emp'],  
        marker='.', ms=4, linestyle="none")  
axs.set_xlabel("Maturity", size=12)  
axs.set_ylabel("Empirical Forward Rate", size=12)  
plt.show()
```



We will consider polynomial models of the form

$$Y_i = \sum_{j=0}^p \beta_j T_i^j + \epsilon_i$$


with a goal of finding the best subset of the p polynomial terms to include in the model. (As is usually the case, we do not consider removing the intercept β_0 .)

The “full” model (largest possible model) we’ll consider includes up to the ninth power.

We will **scale** maturity to have mean zero and variance one:

```
In [12]: forratedat['maturity'] = (forratedat['maturity'] - \
                                     forratedat['maturity'].mean()) / \
                                     (forratedat['maturity'].std())
```

Exercise: Why is scaling the predictor a good idea in this case?

When raise maturity to 9th power, range of values becomes huge, ie. 0.1⁹ to 30⁹, which creates numerical issues for the software

Now, we can fit the full model:

```
In [13]: forratedat['maturity2'] = forratedat['maturity'] ** 2
         forratedat['maturity3'] = forratedat['maturity'] ** 3
         forratedat['maturity4'] = forratedat['maturity'] ** 4
         forratedat['maturity5'] = forratedat['maturity'] ** 5
         forratedat['maturity6'] = forratedat['maturity'] ** 6
         forratedat['maturity7'] = forratedat['maturity'] ** 7
         forratedat['maturity8'] = forratedat['maturity'] ** 8
         forratedat['maturity9'] = forratedat['maturity'] ** 9
```

```
import statsmodels.formula.api as smf

design matrix
→ X_with_ones = sm.add_constant(
    forratedat.drop('emp', axis=1))
    ↖ add column of 1's
    corresponding to  $\beta_0$ 
fullmod = smf.OLS(endog=forratedat['emp'],
    exog=X_with_ones).fit()
    ↖ response
    ↖ design matrix

/Users/chadschafer/anaconda3/lib/python3.7/site-packages/numpy
return ptp(axis=axis, out=out, **kwargs)
```

Exercise: Inspect `fullmod.summary()` and comment on the parameter estimates. Do you think that this model actually needs this many terms, or are we at risk of overfitting?

Including up to 9th power seems excessive. The relationship looks fairly smooth. We are at risk of overfitting.

Exhaustive Search

In cases where the number of predictors is small, an exhaustive search over all possible models is possible.

Exercise: If a model has p candidate predictors, how many models are compared in the exhaustive search?

2^p

```
In [14]: import itertools
         from tqdm import tqdm, tqdm_notebook

X = forratedat.drop('emp', axis=1)
df = {}

for k in tqdm(range(1, len(X.columns)+1)):
    for combo in \
        itertools.combinations(X.columns, k):
        X_with_ones = sm.add_constant(X[list(combo)])
        model = smf.OLS(endog=forratedat['emp'],
                        exog=X_with_ones).fit()
        df[str(combo)] = model.aic

df = pd.Series(df)

HBox(children=(IntProgress(value=0, max=9), HTML(value='')))
```

Exercise: Parse the Python code in the previous example.

X is defined to be the design matrix
without the 1's

`itertools.combinations(X.columns, k)` is a
list of all combinations of size k
of the columns of X

`Af` is a list with all of the AIC
values, with names attached that
provide the included predictors

If you look at `df.idxmin()`, we see that the AIC-optimal model is

$$Y_i = \beta_0 + \beta_1 T_i + \beta_4 T_i^4 + \beta_5 T_i^5 + \beta_6 T_i^6 + \beta_7 T_i^7 + \epsilon_i$$

Let's fit that model.

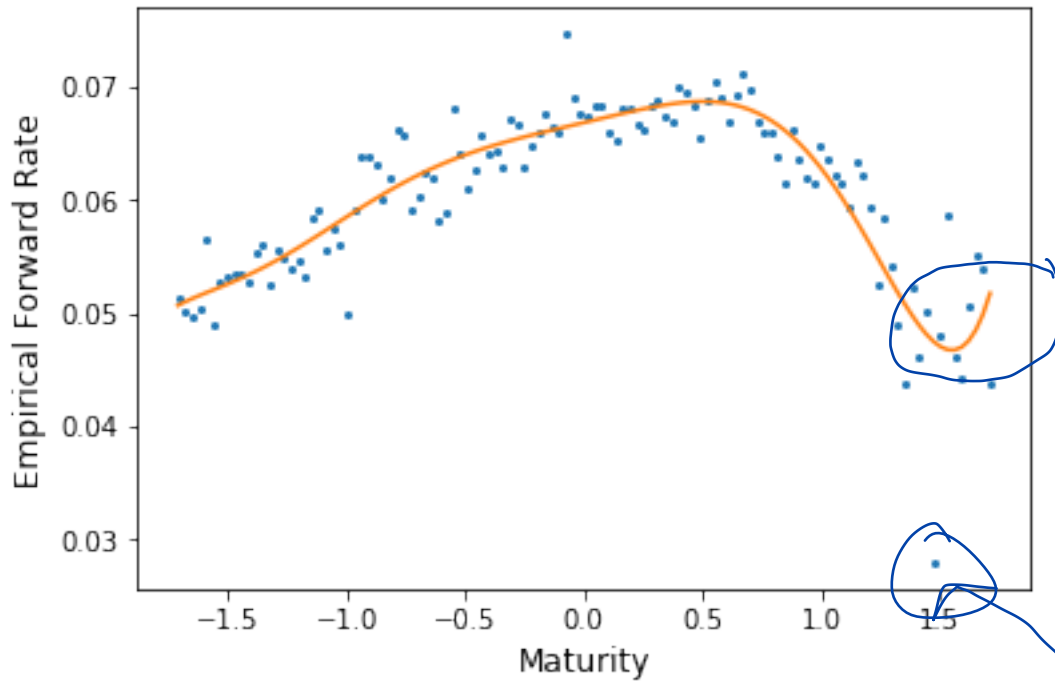
Exercise: Which fit has a larger value of R^2 , the AIC-optimal model, or the full model?

The full model has the largest R^2 among all considered models. R^2 is not a fool for variable selection.

```
In [15]: X_with_ones = \
          sm.add_constant(X.iloc[:, [0, 3, 4, 5, 6]])

AICoptmodel = smf.OLS(\
    endog=forratedat['emp'],
    exog=X_with_ones).fit()

fig, axs = plt.subplots(1, 1, figsize=[6, 4])
axs.plot(forratedat['maturity'], forratedat['emp'],
        marker='.', ms=4, linestyle="none")
axs.set_xlabel("Maturity", size=12)
axs.set_ylabel("Empirical Forward Rate", size=12)
axs.plot(forratedat['maturity'],
        AICoptmodel.fittedvalues, ms=10)
plt.show()
```

Exercise: Comment on the quality of the fit of the model.

The "upturn" on the right side is concerning.
May be overfitting to the one extreme
observation

Stepwise Regression

An exhaustive search is not feasible when there is a large number of predictors.

A classic strategy for dealing with the large number of possible models is to use a **stepwise variable selection** procedure.

With this method, one (typically) starts with the largest model under consideration, and then takes a sequence of **steps**: At each step one predictor is either **added** or **dropped**, depending on if a **forward** or **backward** selection procedure is used, respectively.

The decision for which step to take (i.e., which covariate to add or drop) is based on AIC: You take the step which reduces AIC the most.

Once AIC no longer changes, the process stops, and you have your final model.

This algorithm is far from perfect, but is useful in the appropriate situations.

```

In [16]: features_chosen = []  ← start with null model
    remaining_features = list(X.columns.values)
    best_AIC = np.inf

    for i in range(1, k+1):
        add_feature = False

        for combo in itertools.combinations\
            (remaining_features, 1):
            X_with_ones = sm.add_constant(\
X[list(combo) + features_chosen])
            model = smf.OLS(endog=forratedat\
['emp'], exog=X_with_ones).fit()
            this_AIC = model.aic

            if this_AIC < best_AIC:
                add_feature = True
                best_AIC = this_AIC
                best_feature = combo[0]

    if add_feature:
        features_chosen.append(best_feature)
        remaining_features.remove(best_feature)

```

“forward variable selection”

The resulting model fit is

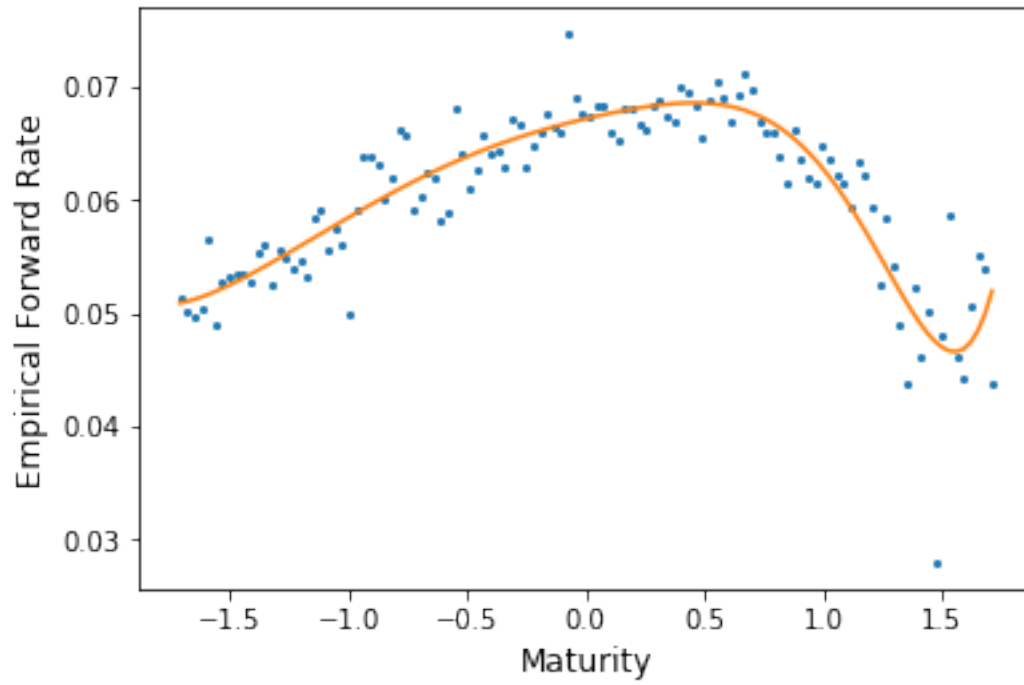
$$Y_i = \beta_0 + \beta_1 T_i + \beta_2 T_i^2 + \beta_4 T_i^4 + \beta_5 T_i^5 + \beta_7 T_i^7 + \beta_8 T_i^8 + \epsilon_i$$

We can see that this is quite similar to the model fit before, although not identical.

```
In [17]: X_with_ones = \
          sm.add_constant(X.iloc[:, [0, 1, 3, 4, 6, 7]])

FSWoptmodel = smf.OLS(\
    endog=forratedat['emp'],
    exog=X_with_ones).fit()

fig, axs = plt.subplots(1, 1, figsize=[6, 4])
axs.plot(forratedat['maturity'], forratedat['emp'],
        marker='.', ms=4, linestyle="none")
axs.set_xlabel("Maturity", size=12)
axs.set_ylabel("Empirical Forward Rate", size=12)
axs.plot(forratedat['maturity'],
        FSWoptmodel.fittedvalues, ms=10)
plt.show()
```



Cross Validation

We've discussed the use of AIC for the purpose of **model selection**, i.e.~the process of determining which covariates should be included in the model.

An important alternative is **leave-one-out cross validation**.

In this approach, one imagines refitting the model n times, each time excluding one of the n observations.

At iteration i , observation i is excluded. The fitted model is then used to predict the response for this observation. Call this $\hat{y}_{(-i)}$.

Finally, we calculate the quantity

$$\text{PRESS} = \sum_{i=1}^n (y_i - \hat{y}_{(-i)})^2$$

where **PRESS** stands for **Prediction Error Sum of Squares**.

The motivation for this is as follows: Once observation i is removed from the fit, we have removed the **influence** that this observation has on the parameter estimates.

Now, we can think of observation i as being a “new” observation, and we can ask: “How well does the model, containing only these covariates, predict the response for this case?”

The quantity $(y_i - \hat{y}_{(-i)})^2$ quantifies the amount of error in this prediction.

Exercise: Which is larger: PRESS or RSS?

$$\text{for each } i \quad \underbrace{(y_i - \hat{y}_{(-i)})^2} \geq \underbrace{(y_i - \hat{y}_i)^2}$$

Adding obs. i into training sample forces the model to try to fit to it, i.e. to choose the parameters to come closer to y_i

See that $\text{PRESS} \geq \text{RSS}$
 $>$

RSS underestimates prediction error

Fortunately, it is not actually necessary to fit n models in order calculate PRESS.

The remarkable result is as follows:

Let $\mathbf{Y}$ be a vector consisting of the n responses, and let $\hat{\mathbf{Y}}$ be a vector consisting of the n fitted values (from the full model).

If it is the case that $\hat{\mathbf{Y}} = \mathbf{H}\mathbf{Y}$, then

$$\underbrace{y_i - \hat{y}_{(-i)}} = \frac{\hat{\epsilon}_i}{1 - h_{ii}}$$

$\hat{\epsilon}_i$ ← residual from full model
← "leverages"

where h_{ii} is the i^{th} diagonal element of $\mathbf{H}$.

In the case of linear regression, it is the case that

$$\mathbf{H} = \mathbf{X} (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$$

where $\mathbf{X}$ whose columns hold the values of the predictors. See Ruppert and Matteson for the details.

This matrix $\mathbf{H}$ is previously mentioned **hat matrix**.

The diagonal elements h_{ii} are called the **leverages**.

The leverages can be found in Python using the following.

```
In [18]: import statsmodels.stats.outliers_influence \
         as outliers_influence

         levs = outliers_influence.OLSInfluence(fullmod)\
         .hat_matrix_diag
```

and then PRESS can be calculated as

```
In [19]: PRESS = ((fullmod.resid / (1 - levs))**2).sum()
         print(PRESS)
```

```
0.0019740282761601747
```

The following code performs an exhaustive search for the optimal model when using PRESS.

```
In [20]: X = forratedat.drop('emp', axis=1)
         df2 = {}

         for k in tnrange(1, len(X.columns)+1):
             for combo in \
                 itertools.combinations(X.columns, k):
                 X_with_ones = \
                     sm.add_constant(X[list(combo)])
                 model = smf.OLS(endog=forratedat['emp'],
                                exog=X_with_ones).fit()
                 levs = outliers_influence.\
                     OLSInfluence(model).hat_matrix_diag
                 df2[str(combo)] = \
                     ((model.resid / (1 - levs))**2).sum()

         df2 = pd.Series(df2)

HBox(children=(IntProgress(value=0, max=9), HTML(value='')))
```

The resulting model fit is

$$Y_i = \beta_0 + \beta_1 T_i + \beta_3 T_i^3 + \beta_4 T_i^4 + \beta_6 T_i^6 + \beta_7 T_i^7 + \epsilon_i$$

This is, again, a slightly different model, but it yields almost identical predictions for the response.

AIC versus PRESS

AIC and PRESS will often give similar, if not the same, result for the model choice.

There are a couple main reasons to prefer PRESS to AIC: First, the quantity being estimated is by PRESS is a very natural measure of prediction performance. Second, the theory behind AIC is based on the assumption that the distributional assumption for the response is correct.

$$AIC = \underline{n \log(RSS)} + \underline{2p}$$

A main reason to choose AIC over PRESS is that AIC is more stable. The estimate that PRESS provides of the expected prediction error is subject to large variance. Hence, PRESS may not perform well for small to moderate sample sizes.

Although PRESS is simple to calculate in the case of linear regression, this will not always be the case. In general, AIC will be easier to calculate than PRESS.

As is often the case, our past experience will often guide our choice of criterion.

Robust Regression

In the least squares approach to regression, we seek the β that minimizes the residual sum of squares

$$\sum_{i=1}^n (y_i - \mathbf{x}_i^T \beta)^2.$$

Of course, this is equivalent to minimizing

$$\frac{1}{\sigma^2} \sum_{i=1}^n (y_i - \mathbf{x}_i^T \beta)^2 = \sum_{i=1}^n \left(\frac{y_i - \mathbf{x}_i^T \beta}{\sigma} \right)^2 = \sum_{i=1}^n \rho \left(\frac{y_i - \mathbf{x}_i^T \beta}{\sigma} \right)$$

where $\rho(x) = x^2$.

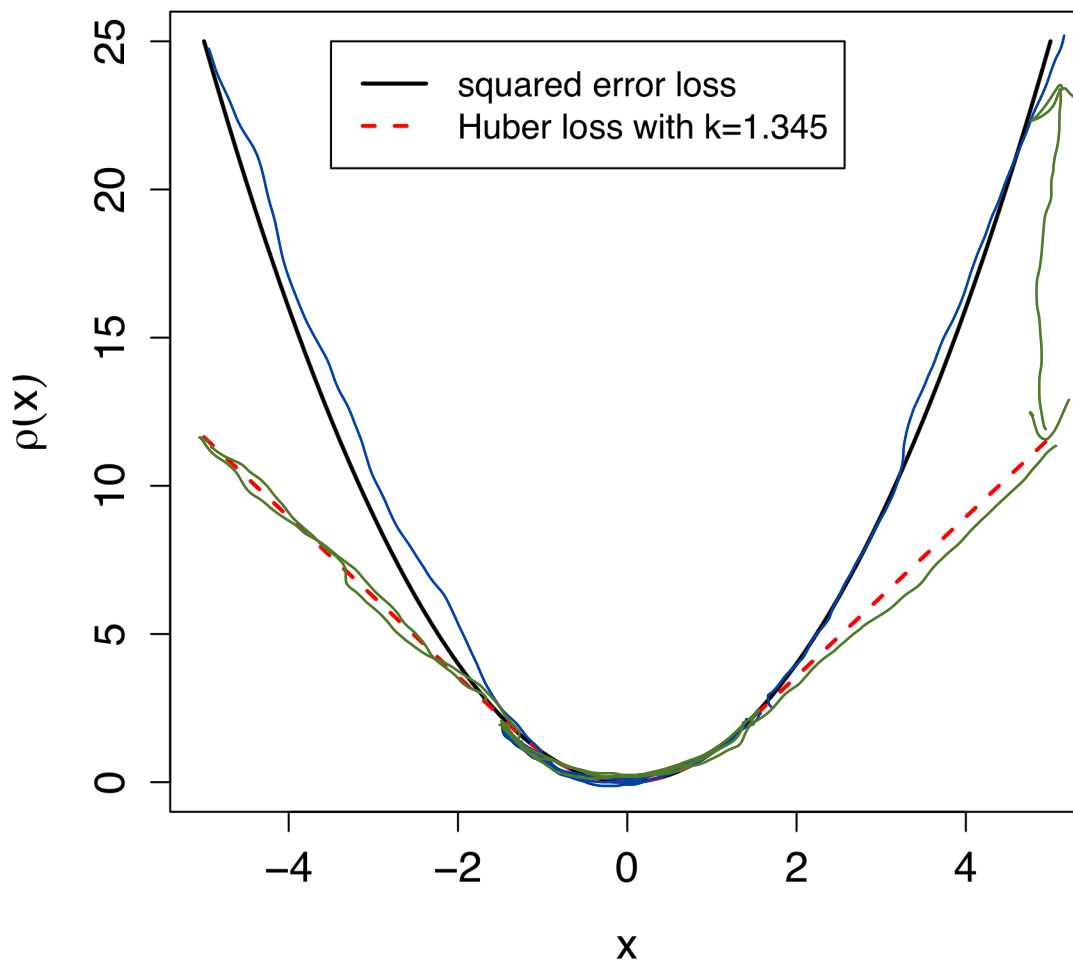
↙ scaled residual

↗ function

The general concern with $\rho(x) = x^2$ is that it may place too much weight on extreme observations: Think about how quickly the function x^2 increases as $|x|$ increases.

The choice $\rho(x) = x^2$ may be optimal when the errors are normal, but it is not very **robust** to deviations from this assumption.

If we generated errors from the t distribution with four degrees of freedom, for instance, we are likely to see a few extreme errors. These observations would have a large influence on the fit of the model, if we proceeded with $\rho(x) = x^2$.



Comparison of Loss Functions.

There are many possible choices for ρ , but here we will focus on a popular one, the **Huber loss function**, which is defined as

$$\rho_{\text{Hub}}(x) = \begin{cases} x^2 & \text{if } |x| < k \\ k(2|x| - k) & \text{otherwise} \end{cases}$$

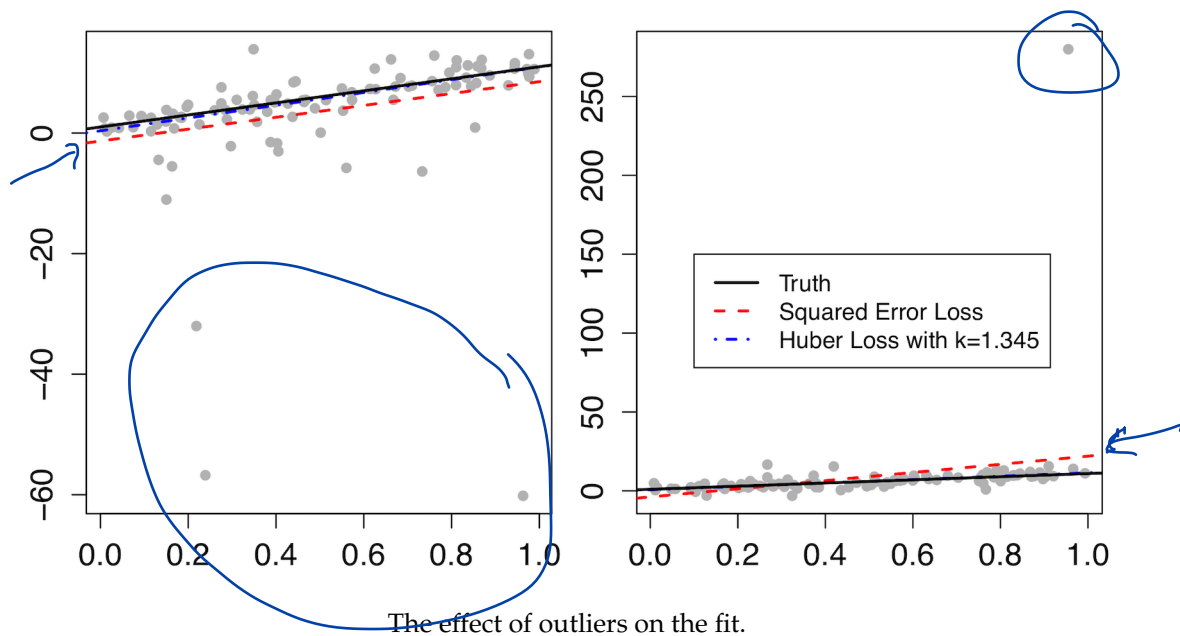
where $k > 0$ is set by the user. For technical reasons, the default choice is $k = 1.345$.

The figure above compares squared error loss with the Huber loss function.

Exercise: What is the effect of varying k in the Huber loss function?

As k increases, estimates converge to least squares

As k decreases, estimates converge to the use of $|x|$ as $\rho(x)$, so-called L1 regression



Example. Here the truth is that $\beta_0 = 1$ and $\beta_1 = 10$. The sample size is $n = 100$. The error has the t distribution with a single degree of freedom, so there is great potential for extreme observations.

The figure above shows two simulated data sets under these conditions, each with the true line (solid), the regression line estimated using least squares (dashed), and the regression line estimated using the Huber loss (dashed-dot).

It is clear that in both instances that the Huber loss function effectively downweights the extreme observations, and, as a result, results in a closer approximation to the truth.

These examples are, admittedly, chosen to be extreme to illustrate the point. Nevertheless, robust regression is useful in situations where it is suspected that the error distribution is not normal, and there is concern over extreme observations.

Robust regression is implemented in Python using RLM from statsmodels.

The argument `M` sets the loss function. (This falls into the general class of problems called **M-estimation**.) The default setting is `M=sm.robust.norms.HuberT`, which uses the Huber loss function with $k = 1.345$.

```
In [21]: ff3modPNCrob = sm.RLM.from_formula\  
          (formula="PNCexret ~ MktRF + SMB + HML",  
           data=ffhold).fit()
```

Exercise: Compare the results of this fit with least squares. Have the coefficients changed much?

Not much of a difference
